Emotion-Aware Agile System for Progress Tracking, Expertise Recommendation, Requirement Management, and Inclusive Communication

25-26J-525

Project Proposal Report

Amarasinghe C N

IT22115348

B.Sc. (Hons) Degree in Information Technology in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

Emotion-Aware Agile System for Progress Tracking, Expertise Recommendation, Requirement Management, and Inclusive Communication

25-26J-525

Project Proposal Report

Amarasinghe C N

IT22115348

Supervisors

Ms. Ishara Weerathunga

Ms. Hansi De Silva

B.Sc. (Hons) Degree in Information Technology in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

August 2025

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	**Signature**
Amarasinghe C N -IT22115348	 .....

Supervisor:

Ms. Ishara Weerathunga

..... 

Date : 30/8/2025.

Abstract

In Agile software development environments, efficient task allocation is critical to maintaining productivity and ensuring balanced workloads across distributed teams. Traditional approaches to task assignment often rely on manual selection or basic keyword matching, which can result in delays, misallocations, and underutilization of expertise. To address this gap, the proposed **AI-Based Expertise Recommendation System** leverages advanced Natural Language Processing (NLP) and machine learning methods to intelligently recommend the most suitable developers for new tasks.

The system integrates task data from **Jira** (issues, backlog items, historical assignments) and **GitHub** (commits, pull requests, file ownership) to build developer expertise profiles based on historical contributions. Using semantic embeddings generated by models such as **BERT** [3] and **CodeBERT** [4], the system captures the contextual meaning of tasks and aligns them with developer-specific embeddings that represent unique domains of expertise (e.g., "authentication specialist," "frontend UI expert") rather than relying on generic skills. A ranking model then computes task-to-developer similarity scores and produces **Top-N ranked recommendations with confidence scores**, improving accuracy and transparency in task assignments.

Unlike conventional tools that rely on keyword similarity or manual lookup [1], this solution emphasizes **semantic understanding, continuous learning through feedback loops, and adaptability to cold-start scenarios** by initializing recommendations from team-level embeddings. The system is designed to operate independently as a standalone recommendation engine, but it can also integrate with Agile workflows to enhance project management.

By improving expertise identification, task allocation, and knowledge sharing, the proposed solution has the potential to reduce assignment delays, optimize workload balance, and increase productivity in Agile teams. This work extends prior research on expertise retrieval [1][2] by combining semantic analysis, ranking mechanisms, and continuous adaptation, making it highly relevant for modern software engineering contexts.

Table of contents

1. **Declaration.....3**
2. **Abstract.....4**
3. **Introduction 6**
4. **Background & Literature survey 8**
    - **4.1 Background..... 14**
    - **4.2 Literature survey.....14**
5. **Research Gap 10**
6. **Research Problem 12**
7. **Objectives 14**
    - **7.1 Main Objectives 14**
    - **7.2Specific Objectives 14**
8. **Methodology..... 16**
9. **Project requirements 19**
    - **9.1 User requirements 19**
    - **9.2 System Requirements 19**
    - **9.3 Functional requirements 20**
    - **9.4 Non-Functional requirements 20**
10. **References 23**

List of figures

Figure 1.1summarizes the comparison between selected existing research systems and the proposed system.....xi
Figure 2.1 Overall System Diagram.....4
Figure 4.1Individual diagram ..... **Error! Bookmark not defined.**

3. Introduction

Agile software development emphasizes iterative progress, collaboration, and adaptive planning. However, one of the persistent challenges faced by Agile teams is the **efficient allocation of tasks to the most suitable developers**. In large or distributed teams, project managers and Scrum Masters often struggle to identify the right expert for a given task, leading to delays, reassignments, and productivity bottlenecks [5]. Traditional assignment methods, such as manual lookup or keyword-based matching, lack semantic depth and often fail to capture the true expertise of developers [6].

To overcome these limitations, recent research in software engineering has explored **expertise retrieval and recommendation systems**. Such systems analyze historical data from version control platforms (e.g., GitHub) and issue tracking tools (e.g., Jira) to infer developer expertise [5][6]. By leveraging **Natural Language Processing (NLP)** and **machine learning** techniques, expertise recommendation can be made more intelligent, moving beyond surface-level keyword matching toward semantic understanding of tasks and developer skills [7].

The proposed **AI-Based Expertise Recommendation System** builds upon these advances by creating **developer expertise profiles** from historical contributions such as commits, pull requests, and resolved issues. Using embedding models like **BERT** and **CodeBERT**, task descriptions and developer history are converted into semantic vectors, enabling the system to compute similarity scores between new tasks and developer expertise domains [7][8]. A ranking algorithm then generates **Top-N developer recommendations with confidence scores**, allowing project managers to make informed decisions about task assignments.

This approach not only improves task allocation accuracy but also enhances **knowledge sharing, workload balancing, and collaboration within Agile teams**. Furthermore, the system is designed to handle **cold-start scenarios** by leveraging semantic similarity and team-level embeddings, making it applicable even when historical data is sparse.

4.Background and Literature survey

4.1 Background

Agile teams depend on fast, accurate task assignment to sustain velocity and quality. In practice, identifying “the right developer for the right task” is hard because expertise is distributed, codebases evolve, and knowledge is embedded in issue trackers, version control, and review discussions. Early approaches relied on manual lookup or keyword matching over bug reports and source files, which often missed contextual signals like file ownership, module history, and reviewer feedback [9][10][11]. Subsequent work showed that incorporating developer activity traces (commits, reviews, pull requests) and socio-technical factors (ownership, experience) improves prediction quality for assignees and change request routing [12][13][14].

Modern expertise recommenders increasingly treat task assignment as an information-retrieval or ranking problem: represent a task and each developer’s historical work with features, compute relevance, and rank candidates [15][19]. Representations evolved from TF-IDF/BM25 term vectors to distributed embeddings (Doc2Vec) and, more recently, transformer-based encoders (BERT) that capture semantics beyond surface keywords [15][16][17]. For software artifacts specifically, **CodeBERT** and similar code+NL models better encode identifiers, APIs, and code contexts, enabling more faithful matches between task descriptions and expertise profiles [18]. These advances support practical needs in Agile settings: quicker assignments, reduced rework, better workload balance, and improved knowledge sharing.

4.2 Literature Survey

Early bug triage and expert finding. Foundational studies framed bug triage as classification or IR, assigning reports to likely fixers using textual similarity and developer history [9][11]. Enterprise "expertise recommenders" formalized expert finding as ranking candidates by evidence of competence in a topic area [10][19]. These works established core signals: prior fixes, topical similarity, and communication traces.

Change-request routing and socio-technical factors. Later research integrated repository mining and dependency awareness to route change requests, highlighting that **code ownership and contributor experience** correlate with quality and are strong predictors for assignment [12][13][20]. Analyses of the **pull-based model** on GitHub emphasized the value of PR discussions, review outcomes, and file paths as features for expertise inference [14].

From lexical to semantic representations. Traditional TF-IDF/BM25 retrieval remains competitive and interpretable for textual artifacts [15], but **distributed representations** improved robustness to vocabulary mismatch. **Doc2Vec** enabled task and developer embeddings based on historical documents [16]. **Transformer models** (BERT) further advanced semantic matching in natural language tasks by encoding context and polysemy [17]. For software engineering, **CodeBERT** fused natural and programming languages, outperforming text-only models on code-related understanding and retrieval [18].

Ranking and learning-to-rank. Many systems cast assignment as ranking: compute task–developer similarity (text, files, topics, ownership) and train a model to order candidates using historical assignee labels [9][11][19]. This supports **Top-N recommendation with confidence**, aligning with real Agile workflows where multiple viable assignees exist.

Gaps motivating this component. Despite progress, practical gaps persist: (i) limited semantic understanding of mixed code/issue text, (ii) brittle keyword reliance in heterogeneous teams, (iii) weak handling of **cold-start** developers or new

modules, and (iv) insufficient feedback loops to adapt as projects evolve. Your component addresses these by (a) using **semantic (BERT/CodeBERT) embeddings** for tasks and developer histories, (b) enriching signals with **file-path overlap, language match, ownership**, and (c) employing **continuous learning** from acceptance vs. override of recommendations.

5. Research Gap

Although expertise recommendation has been studied in the context of bug triage, change request routing, and expert finding, significant limitations remain that hinder adoption in real-world Agile environments.

1. **Keyword Dependency**: Early bug triage approaches [21][23] rely heavily on keyword or vocabulary-based similarity, which often fails when task descriptions use different wording from historical issues.

2. **Limited Context Awareness**: Many existing systems [24][26] incorporate socio-technical factors like file ownership or developer activity but lack semantic understanding of task descriptions, especially when mixing natural language and source code.

3. **Cold-Start Problem**: Traditional systems struggle to recommend suitable developers for new modules or recently joined team members with little history [22][31].

4. **Static Models without Feedback**: Most research systems [25][28] train on historical data but lack continuous learning mechanisms to adapt as projects, technologies, or team expertise evolve.

5. **Lack of Confidence Scoring & Ranking**: Several models return a single predicted assignee [21][23], which reduces flexibility. Agile workflows often require **Top-N ranked suggestions** with confidence levels to support human decision-making.

6. **Integration with Agile Workflows**: Existing approaches rarely align directly with Scrum or Kanban practices. They provide recommendations in isolation without integration into Agile tools (e.g., Jira, GitHub) [26].

The **proposed system** addresses these gaps by:

- Using **semantic embeddings (BERT/CodeBERT)** to capture contextual meaning of tasks and developer expertise.
- Incorporating **multi-feature signals** (task embeddings, file-path overlap, programming language match, ownership history).
- Supporting **cold-start developers** via team-average embeddings and similarity-based initialization.
- Enabling **continuous learning** with feedback loops from accepted/rejected recommendations.
- Producing **Top-N ranked developer suggestions with confidence scores**, making it more actionable for Scrum Masters.

Feature / Approach	Anvik et al. (2006) [1]	Matter et (2009) [3]	Kagdi et al. (2012) [4]	Gousios et al. (2014) [6]	Proposed System
Data Sources	Bug reports (text)	Vocabulary-based bug reports	Change requests repository p.	GitHub PRs & discussions	Jira issues + GitHub commits PRs, ownership
Representation	TF-IDF keywords	Vocabulary models	Keyword + file ownership	Social + technical factor	Semantic emboddings (BERT/CodeBERT)
Recommendation Output	Single developer	Single developer	Candidate set	Candidate set	Top-N ranked experts with confidence
Cold-Start Handling	✗	✗	⚠	⚠	✓
Learning Approach	Classification	IR-based	Feature-based retrieval	Empirical analysis	Continupus feedback + retraining
Feedback/Adaptation	✗	✗	⚠	✓	✓
Agile Integration	✗	✗	Weak	Weak	Strong (direct Jira/GitHub integration)

Figure 1.1summarizes the comparison between selected existing research systems and the proposed system

Figure 1.1 compares existing expertise recommendation approaches with the proposed system across seven key dimensions. The analysis highlights several limitations in prior work and shows how the proposed system addresses them.

- **Data Sources**: Earlier studies relied primarily on bug reports or change requests, which limited the scope of available information. The proposed system extends beyond text-based reports by incorporating Jira issues, GitHub commits, pull requests, and ownership history, thereby providing a richer and more comprehensive context for expertise modeling.

- **Representation**: Traditional approaches used keyword- or vocabulary-based methods (e.g., TF-IDF), which fail to capture semantic meaning. Later work added socio-technical signals, but lacked deep contextual understanding. The proposed system leverages semantic embeddings (BERT/CodeBERT) to model both natural language descriptions and source code, enabling more accurate task–expert matching.

- **Recommendation Output**: Early systems produced a single assignee, which reduced flexibility. Some later approaches expanded to candidate sets but without prioritization. The proposed system generates **Top-N ranked recommendations with confidence scores**, allowing Scrum Masters and Agile teams to make informed decisions.

- **Cold-Start Handling**: Most prior research did not address cold-start scenarios, where new modules or developers with little history are involved. The proposed system mitigates this by initializing new developers with team-average embeddings and similarity-based strategies, ensuring meaningful recommendations from the outset.

- **Learning Approach**: Prior systems relied on static models (classification, IR-based retrieval, or empirical analysis) trained on historical data. These models quickly become outdated as projects evolve. The proposed system incorporates a **continuous learning framework**, retraining on user feedback and adapting to shifts in team expertise and project technologies.

- **Feedback and Adaptation**: Feedback was either absent or minimally considered in earlier work. The proposed approach integrates explicit **feedback loops**, where accepted or rejected recommendations are used to improve subsequent predictions.

- **Agile Integration**: Existing approaches rarely aligned with Agile workflows and provided recommendations in isolation from team tools. The proposed system ensures **direct integration with Jira and GitHub**, enabling seamless adoption within Scrum and Kanban practices.

6.Research Problem

Agile software development has become one of the most widely adopted methodologies in modern software engineering, enabling teams to collaborate, adapt, and deliver software efficiently. However, while Agile emphasizes transparency and communication, it often overlooks the **emotional dynamics** of team members. Emotions such as frustration, stress, burnout, or demotivation frequently surface in team interactions through chat platforms or virtual meetings, yet these signals are rarely detected or acted upon in real time. Ignoring these emotional cues can lead to decreased productivity, conflicts, and poor sprint outcomes, which undermine Agile principles of collaboration and continuous improvement.

Existing Agile management tools are primarily **task- and metric-oriented**, focusing on backlog tracking, sprint velocity, and defect rates, but they lack mechanisms to capture and interpret the **human factors** driving these outcomes. Although some emotion detection tools exist, they are often limited to a single modality (text or facial expression) and are not integrated into Agile workflows. Moreover, they provide descriptive insights without offering **decision-support recommendations** that Scrum Masters and Product Owners can act upon.

This creates a **research gap** in developing an **emotion-aware Agile assistant** capable of integrating multimodal emotion detection (text, facial expressions, micro-expressions, and voice cues) with Agile metrics. Such a system could not only identify emotional states but also **predict their impact on sprint performance** and recommend proactive interventions, such as workload redistribution or conflict resolution. Addressing this problem is critical for enhancing collaboration, maintaining team morale, and ensuring sustainable productivity in distributed Agile teams.

7. Objectives

7.1. Main Objective

The primary objective of this research is to design and implement an AI-Based Expertise Recommendation System that intelligently recommends the most suitable developers for new tasks within Agile teams. The system will leverage semantic embeddings from task descriptions (such as Jira issues, GitHub commits, and project documentation) and match them with developer expertise profiles built from their historical contributions. Unlike traditional keyword-based task allocation, this system will analyze the contextual meaning of tasks, generate developer-specific expertise representations, and provide ranked recommendations with confidence scores. By doing so, the system ensures more accurate task assignments, improves workload balancing, reduces delays in project execution, and enhances team productivity.

7.2. Specific Objectives

1. **Emotion-Aware Agile Monitoring System**
   To monitor team communications across platforms (Slack, Jira, GitHub, and video meetings) and detect emotions such as stress, burnout, or motivation using Natural Language Processing and facial expression recognition. The insights generated will be used to understand how emotional factors influence productivity and collaboration, providing additional context to task allocation and sprint planning.

2. **Requirement Change Impact & Sprint Replanning Tracker**
   To collect and analyze backlog and sprint requirement data to identify changes and predict their impact on sprint performance. This component will provide proactive recommendations for workload redistribution, requirement adjustment, and sprint replanning, ensuring that developer expertise recommendations align with evolving project conditions.

3. **Inclusive Brainstorming & Communication Empowerment Platform**
   To enable fair and inclusive participation in team discussions by transforming hesitant, fragmented, or multilingual inputs into clear, professional communication. This component will ensure that ideas from all team members, including those less confident in expressing themselves, are considered during task assignments and project planning. By promoting inclusivity, this module supports more diverse and effective expertise utilization.

8. Methodology

The proposed system follows a multi-layered methodology that integrates communication analysis, task allocation intelligence, requirement tracking, and inclusive interaction support into a unified framework for Agile project management. The process begins with the collection of data from diverse sources, including developer communication logs, project management tools, and collaborative platforms. This information is then preprocessed to remove noise, standardize formats, and ensure that both textual and non-textual signals can be analyzed consistently.

Once prepared, the data flows through several specialized analytical modules. One stream focuses on understanding the emotional state of the development team by analyzing natural language patterns and facial expressions, while another concentrates on interpreting project tasks and mapping them to relevant expertise. In parallel, changes in sprint requirements and backlog items are monitored, with predictive models assessing their potential impact on productivity, velocity, and delivery risks. Alongside these analytical processes, a communication enhancement layer refines fragmented or hesitant inputs, ensuring that every team member has an equal opportunity to contribute meaningfully to the decision-making process.

The insights generated from these streams are then consolidated into an integration layer that balances emotional awareness, expertise recommendations, requirement risks, and inclusive driven but also emotionally and socially aware, making project management more adaptive and effective.collaboration. Finally, the processed outcomes are presented in the form of dashboards and proactive alerts, providing Scrum Masters and team leaders with actionable intelligence. This methodological

flow ensures that project management decisions are both data-driven and human-centric, promoting productivity, fairness, and sustainability within Agile teams.

- **Emotion Monitoring** analyzes textual communication and facial expressions to detect emotions such as frustration, burnout, or motivation.
- **Expertise Recommendation** processes task descriptions and developer histories to recommend the most suitable experts for new assignments.
- **Requirement Change & Sprint Replanning Tracker** monitors backlog changes and predicts their impact on sprint performance, providing suggestions for dynamic replanning.
- **Inclusive Brainstorming & Communication Empowerment** ensures equal participation by refining hesitant or multilingual inputs and presenting them clearly for team consideration.

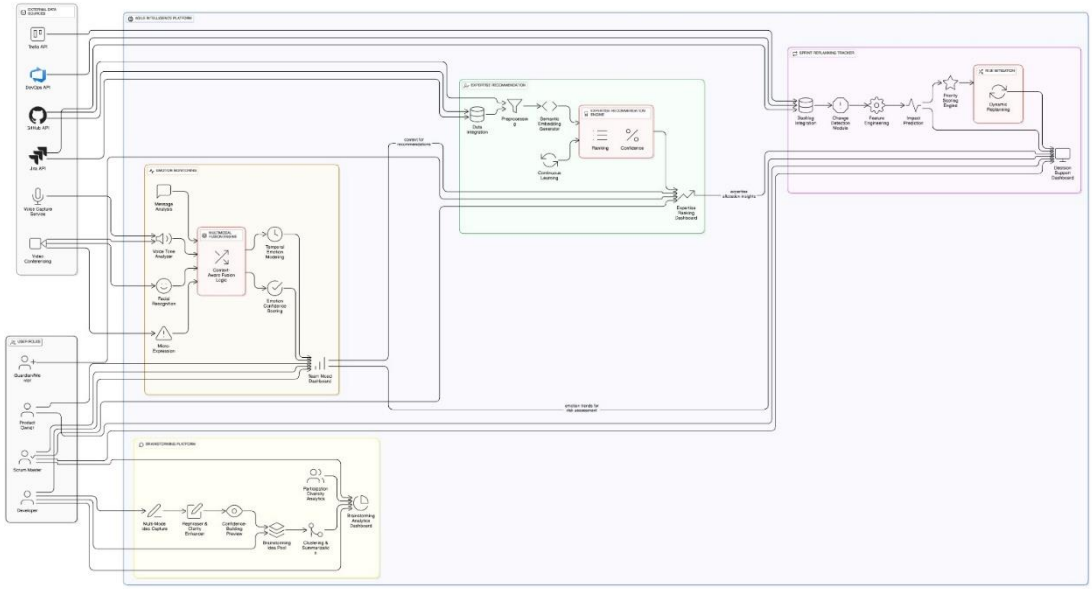

Figure 2.1 Overall System Diagram

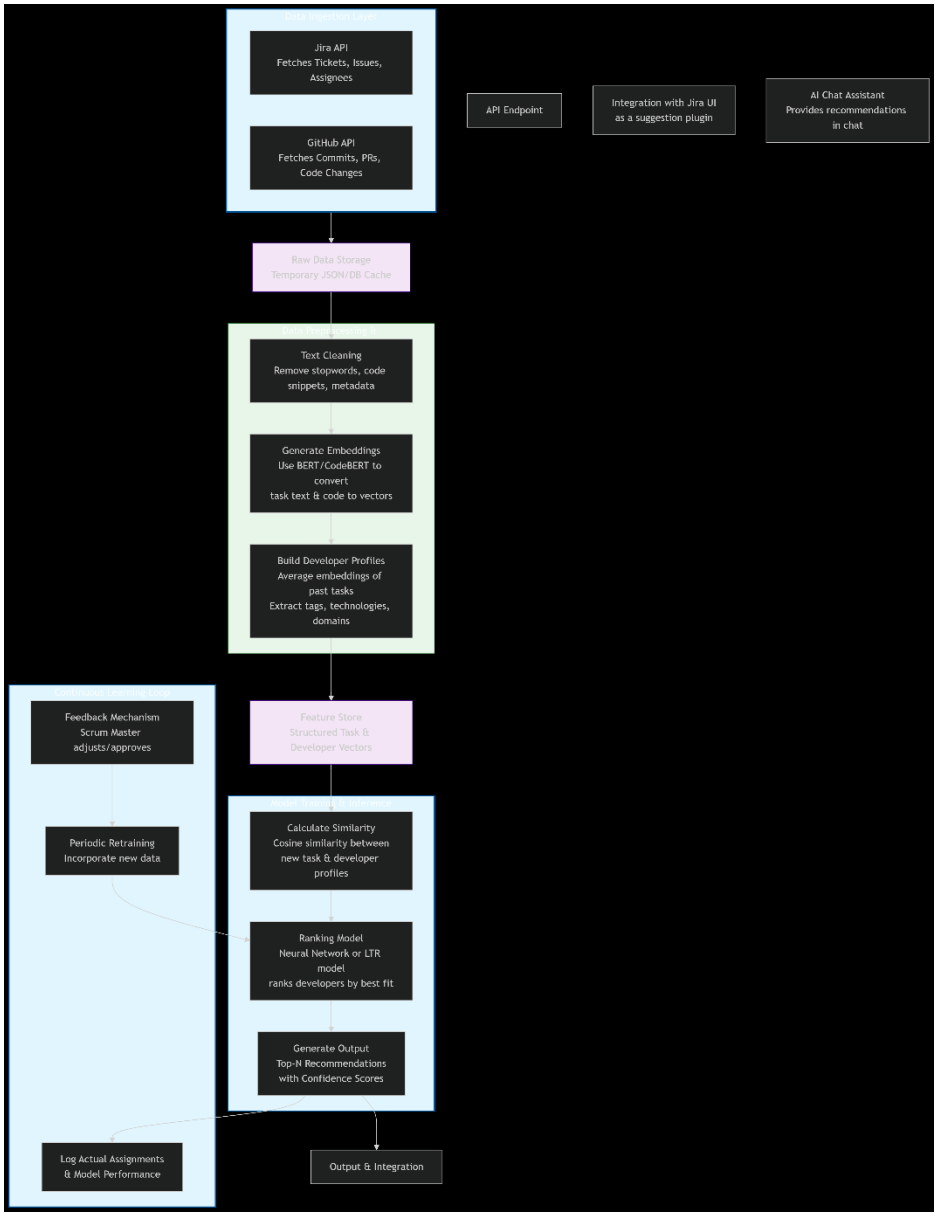

Figure 3.1Individual diagram

The **Expertise Recommendation System** focuses on optimizing task allocation in Agile projects by identifying the most suitable developers for each new task. The methodology follows a structured pipeline:

1. **Data Collection**
   Project-related data is extracted from sources such as Jira (issues, tickets, story descriptions) and GitHub (commits, pull requests, file changes). Developer contribution histories are also recorded to build expertise profiles.

2. **Data Preprocessing & Feature Engineering**
   The collected data is cleaned by removing stopwords, irrelevant metadata, and code snippets. Task descriptions and developer contributions are then transformed into **semantic embeddings** using advanced models (e.g., BERT, CodeBERT). This ensures that the system captures contextual meaning rather than relying on simple keyword matching.

3. **Expertise Profiling & Similarity Computation**
   Developer expertise vectors are generated from historical contributions and compared with new task embeddings. A **similarity score** is calculated to determine the degree of fit between the task and each developer.

4. **Recommendation Model & Ranking**
   A ranking algorithm evaluates all candidates and produces **Top-N ranked recommendations** along with confidence scores. This allows Scrum Masters to make informed task assignments by considering both expertise and system confidence.

5. **Integration & Decision-Support**
   The recommendations are presented on the system dashboard and cross-validated with other components. For example, workload balance insights from Component 3 and emotional states from Component 1 are factored in to avoid overloading stressed developers.

9. Project Requirements

9.1. User Requirements

The system is intended to support Agile teams, including developers, Scrum Masters, and Product Owners. From a user perspective, the requirements are as follows:

- The system should allow users to log in securely and access project-related recommendations.
- Scrum Masters and Product Owners should be able to view task recommendations with ranked developer suggestions and confidence levels.
- Developers should have visibility into their expertise profiles and receive feedback on tasks assigned to them.
- Users should be able to interact with the system through a simple and intuitive dashboard without needing technical expertise in AI.
- Alerts and recommendations should be delivered in real time to support quick decision-making during sprint planning and execution.

9.2. System Requirements

Hardware Requirements

- Server with sufficient computational power to handle NLP and embedding models (minimum 16 GB RAM, multi-core processor, and GPU support for model training).
- Reliable storage to maintain historical project and communication data (minimum 200 GB).
- Client-side devices (PCs/laptops) with standard browsers for accessing the system dashboard.

Software Requirements

- Backend built using Node.js/Express or Python-based services for data processing.

- Database (e.g., MongoDB or PostgreSQL) for storing developer profiles, task embeddings, and recommendations.
- Machine learning libraries and frameworks (TensorFlow, PyTorch, Hugging Face Transformers) for embedding generation and ranking models.
- APIs for integration with Jira, GitHub, Slack, or other Agile tools.
- Web-based dashboard frontend (React or Angular).

9.3. Functional Requirements

The system should provide the following functionalities:

1. Collect task descriptions, commits, and issue histories from integrated Agile tools.
2. Preprocess text data by cleaning and converting it into meaningful representations.
3. Generate developer expertise profiles using historical contribution data.
4. Match new tasks with developer profiles using semantic embeddings and similarity measures.
5. Provide ranked recommendations of developers for each task with confidence scores.
6. Update recommendations continuously as new project data is collected.
7. Deliver recommendations through an interactive dashboard for Scrum Masters and Product Owners.
8. Allow users to give feedback on recommendations to improve the system over time.
9. Ensure recommendations are context-aware by considering workload balance and evolving sprint requirements.

9.4. Non-Functional Requirements

The system should also meet the following quality standards:

- **Performance**: Recommendations should be generated within a few seconds to support real-time Agile planning.

- **Scalability**: The system must handle multiple teams and projects simultaneously without performance degradation.
- **Accuracy**: Recommendation accuracy should improve over time through continuous learning and feedback.
- **Security**: All project and developer data must be stored securely with role-based access control and data encryption.
- **Reliability**: The system should maintain high availability (99% uptime) to ensure it is always accessible during sprint activities.
- **Usability**: The dashboard interface should be user-friendly, requiring minimal training for adoption by Agile teams.
- **Maintainability**: The system architecture should support easy updates and integration of improved machine learning models.

10. References

[1] Baltes, S., et al. (2019). *Who should fix this bug? Automatically recommending developers*. Empirical Software Engineering.

[2] Gousios, G., Pinzger, M., & Deursen, A. (2016). *An exploratory study of the pull-based software development model*. ICSE.

[3] Vaswani, A., et al. (2017). *Attention is all you need*. NeurIPS.

[4] Feng, Z., et al. (2020). *CodeBERT: A pre-trained model for programming and natural languages*. EMNLP.

[5] Baltes, S., et al. (2019). *Who should fix this bug? Automatically recommending developers*. Empirical Software Engineering.

[6] Gousios, G., Pinzger, M., & Deursen, A. (2016). *An exploratory study of the pull-based software development model*. ICSE.

[7] Vaswani, A., et al. (2017). *Attention is all you need*. NeurIPS.

[8] Feng, Z., et al. (2020). *CodeBERT: A pre-trained model for programming and natural languages*. EMNLP.

[9] Anvik, J., Hiew, L., Murphy, G.C. (2006). *Who Should Fix This Bug?* ICSE.

[10] McDonald, D.W., Ackerman, M.S. (2000). *Expertise Recommender*. CSCW.

[11] Matter, H., Kuhn, A., Nierstrasz, O. (2009). *Assigning Bug Reports using a Vocabulary-Based Expertise Model*. MSR.

[12] Kagdi, H., Malik, Z., Mahmoud, A. (2012). *Who Can Help Me with This Change Request?* ICSM.

[13] Bird, C., Nagappan, N., Murphy, B., Gall, H., Devanbu, P. (2011). *Don't Touch My Code! Examining the Effects of Ownership on Software Quality*. ESEC/FSE.

[14] Gousios, G., Pinzger, M., van Deursen, A. (2014). *An Exploratory Study of the Pull-based Software Development Model*. FSE.

[15] Robertson, S., Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. FnTIR.

[16] Le, Q., Mikolov, T. (2014). *Distributed Representations of Sentences and Documents*. ICML.

[17] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL.

[18] Feng, Z., et al. (2020). *CodeBERT: A Pre-Trained Model for Programming and Natural Languages*. EMNLP.

[19] Balog, K., et al. (2012). *Expertise Retrieval*. FnTIR.

[20] Rahman, F., Devanbu, P. (2011). *Ownership, Experience and Defect-Proneness in Software*. ICSE.

[21] Anvik, J., Hiew, L., Murphy, G.C. (2006). *Who Should Fix This Bug?* ICSE.

[22] McDonald, D.W., Ackerman, M.S. (2000). *Expertise Recommender*. CSCW.

[23] Matter, H., Kuhn, A., Nierstrasz, O. (2009). *Assigning Bug Reports using a Vocabulary-Based Expertise Model*. MSR.

[24] Kagdi, H., Malik, Z., Mahmoud, A. (2012). *Who Can Help Me with This Change Request?* ICSM.

[25] Bird, C., Nagappan, N., Murphy, B., Gall, H., Devanbu, P. (2011). *Don't Touch My Code! Examining the Effects of Ownership on Software Quality*. ESEC/FSE.

[26] Gousios, G., Pinzger, M., van Deursen, A. (2014). *An Exploratory Study of the Pull-based Software Development Model*. FSE.

[27] Robertson, S., Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. FnTIR

[28] Le, Q., Mikolov, T. (2014). *Distributed Representations of Sentences and Documents*. ICML.

[29] Devlin, J., et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL.

[30] Feng, Z., et al. (2020). *CodeBERT: A Pre-Trained Model for Programming and Natural Languages*. EMNLP.

[31] Balog, K., et al. (2012). *Expertise Retrieval*. FnTIR.